Universidad Tecnológica del Perú

DISEÑO DE PATRONES

MOTIVACIÓN: Creando una historia

- Cada estudiante dirá una frase para crear una historia en conjunto.
- El alumno que diga la primera frase deberá señalar a un compañero que continuará la historia; si no conoce sus nombres, puede señalar un color de prenda. (“Continúa mi compañero con polo verde”).
- La historia inicia así: *“En los montes de la selva peruana, existía…”*.

<u>Consigna</u>

- Debes hacer lo posible por mantener coherencia con la historia.
- Tienes total libertad creativa, ¡deja volar tu imaginación!

Universidad Tecnológica del Perú

CONOCIMIENTOS PREVIOS

¿Qué entiendes por patrones?

LOGRO DE LA SESIÓN

Al finalizar la unidad el participante identifica los principios de diseño de los patrones de software para la solución de problemas

IMPORTANCIA DE LA SESIÓN

La aplicación de los patrones de software a permitirá solucionar los problemas habituales de software planteados.

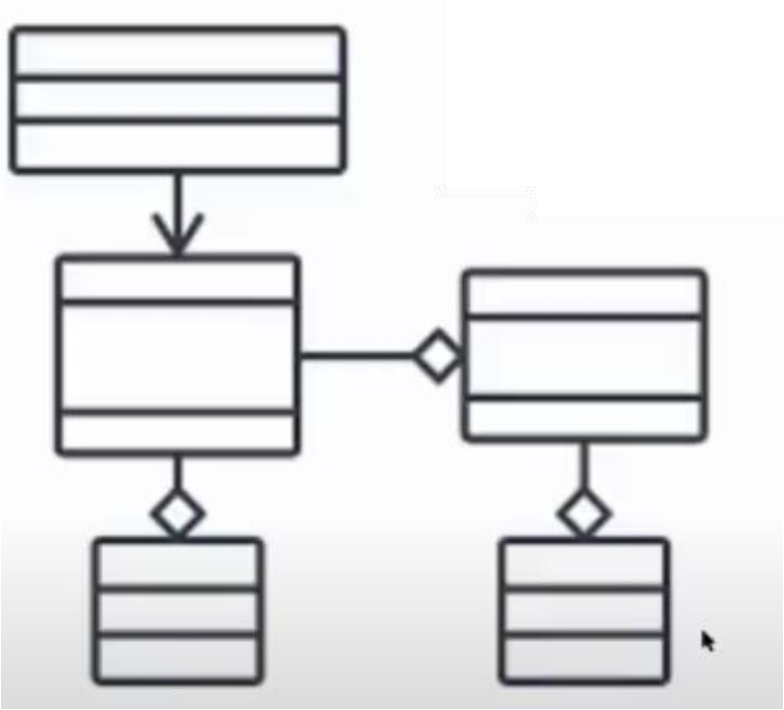
TEMA 1:

- Introducción a los Patrones, Concepto,Historia.
- Patrones arquitectónicos vs Patrones de Diseño.
- Patrones GOF: Beneficios, cuando y como usarlos.
- Clasificación según propósito: creaciones, estructurales y comportamiento.
- Especificación de Patrones de Diseño.

Introducción a los patrones

¿ Que es un Patrón?

Un patrón es una solución a un problema en un contexto



Introducción a patrones

¿ Que es un Patrón diseño?

Son soluciones reutilizables a problemas comunes en el desarrollo de software. Proporcionan una manera estandarizada de resolver problemas que se presentan frecuentemente en el diseño de sistemas, promoviendo buenas prácticas y facilitando la comunicación entre desarrolladores.

Introducción a patrones

Historia:

- **1970s:** El concepto de patrones se popularizó con el libro *"A Pattern Language"* de Christopher Alexander, que aplicó patrones a la arquitectura y el urbanismo.

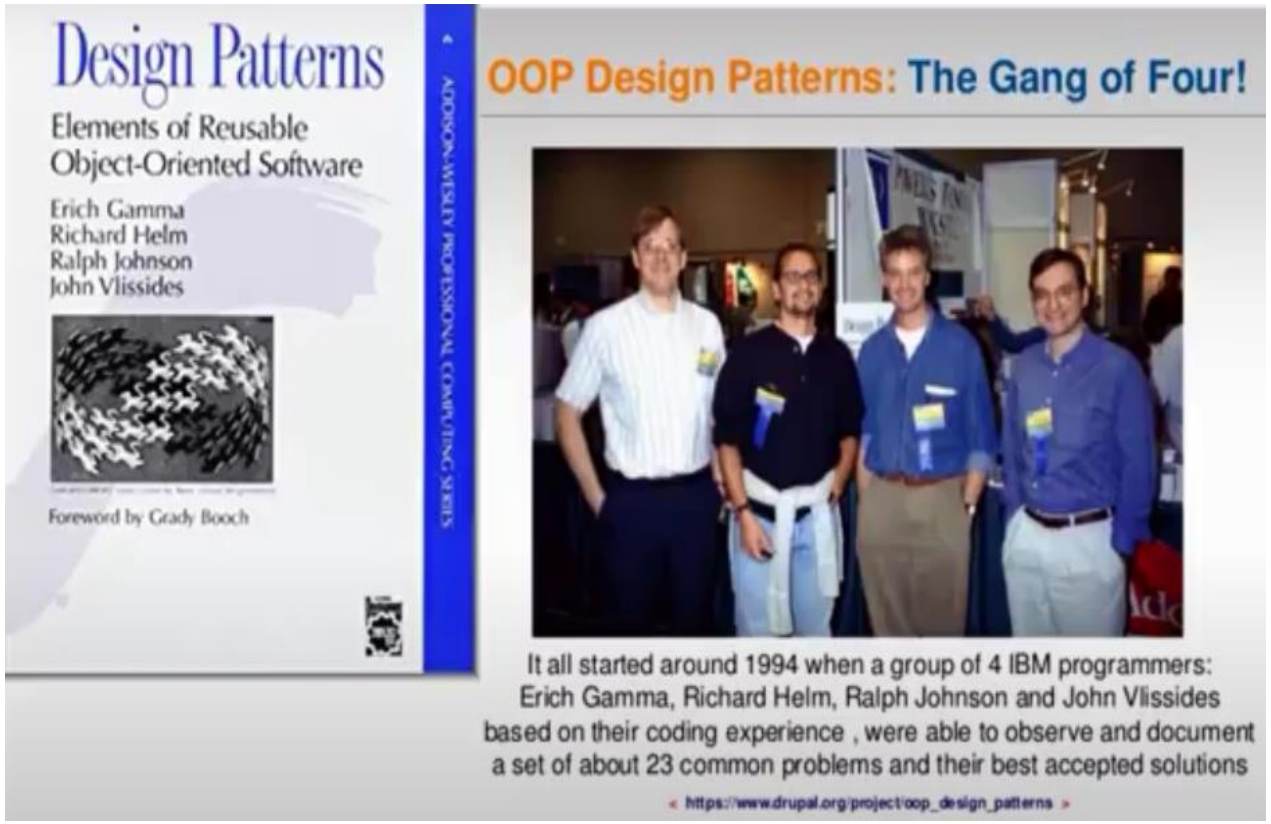
- **1994:** *"Design Patterns: Elements of Reusable Object-Oriented Software"* por Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides, conocidos como los "Gang of Four" (GOF), adaptó los patrones a la programación orientada a objetos, estableciendo una base para los patrones de diseño en software.

- .

Introducción a patrones

En 1994 apareció el libro Design Patterns: Elements of Reusable Object Oriented Software, escrito por los ahora famosos Gang of Four (GOF)

Gof: Erich Gamma, richard Helm, Ralph Johnson y John Vlissides



Universidad Tecnológica del Perú

Los patrones Pretenden

- Proporcionar catálogos de elementos reusables en el diseño de sistemas software
- Evitar la reiteración en la búsqueda de soluciones a problemas ya conocidos y solucionados anteriormente.
- Formalizar un vocabulario común entre diseñadores.
- Estandarizar el modelo en que se realiza el diseño
- Facilitar el aprendizaje de las nuevas generaciones de diseñadores condensando conocimiento existente.

Los patrones NO pretenden

- Imponer ciertas alternativas de diseño frente a otras.
- Eliminar la creatividad inherente al proceso de diseño

No es obligatorio utilizar los patrones

Es aconsejable en el caso de tener el mismo problema o similar que soluciona el patron.

Abusar o forzar el uso de los patrones puede ser un error.

¿ Por que debo usar patrones?

- Los patrones de diseño son soluciones bien pensadas a problemas conocidos de programación.
- Muchos programadores han padecido de estos problemas antes y han utilizado estas Soluciones par remedio
- No reinventar la rueda.

Universidad Tecnológica del Perú

Patrones Arquitectónicos VS Patrones de Diseño

Patrones Arquitectónicos: Se centran en la estructura global de un sistema y cómo sus componentes interactúan.

- **Ejemplos:** Modelo-Vista-Controlador (MVC), Arquitectura en Capas.
- **Uso:** Definen la organización general y la distribución de las responsabilidades en un sistema.

Universidad Tecnológica del Perú

Patrones Arquitectónicos VS Patrones de Diseño

Patrones de Diseño: Abordan problemas más específicos en el diseño de clases y objetos, ayudando a diseñar sistemas modulares y reutilizables.

- **Ejemplos:** Singleton, Factory Method, Observer.
- **Uso:** Proporcionan soluciones a problemas recurrentes en el diseño de software.

Patrones GOF (Gang of Four):

Son una colección de 23 patrones de diseño orientados a objetos, documentados en el libro "Design Patterns: Elements of Reusable Object-Oriented Software" por Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides.

Estos patrones ofrecen soluciones estandarizadas y reutilizables para problemas comunes en el diseño de software, mejorando la eficiencia y mantenibilidad del código.

Se dividen en tres categorías: Creacionales, Estructurales y de Comportamiento.

Patrones GOF (Gang of Four): Colección

1.Patrones Creacionales: Abordan la creación de objetos.

- **Singleton:** Asegura que una clase tenga una única instancia.
- **Factory Method:** Define una interfaz para crear un objeto, pero permite a las subclases decidir qué clase instanciar.
- **Abstract Factory:** Proporciona una interfaz para crear familias de objetos relacionados.
- **Builder:** Separa la construcción de un objeto complejo de su representación.
- **Prototype:** Permite crear nuevos objetos copiando una instancia existente.

DISEÑO DE PATRONES

Patrones GOF (Gang of Four): Colección

2. **Patrones Estructurales:** Abordan la composición de clases y objetos.
   - **Adapter:** Permite que clases con interfaces incompatibles trabajen juntas.
   - **Bridge:** Desacopla una abstracción de su implementación.
   - **Composite:** Compone objetos en estructuras de árbol para representar jerarquías.
   - **Decorator:** Añade responsabilidades a objetos de manera dinámica.
   - **Facade:** Proporciona una interfaz simplificada a un subsistema complejo.
   - **Flyweight:** Usa el compartimiento para soportar un gran número de objetos de grano fino de manera eficiente.
   - **Proxy:** Proporciona un sustituto o marcador de posición para otro objeto

DISEÑO DE PATRONES

Patrones GOF (Gang of Four): Colección

3. **Patrones de Comportamiento:** Abordan la comunicación entre objetos.

- **Chain of Responsibility:** Permite que más de un objeto maneje una petición.
- **Command:** Encapsula una petición como un objeto.
- **Interpreter:** Define una representación para una gramática y un intérprete para el lenguaje.
- **Iterator:** Proporciona una forma de acceder a los elementos de un objeto agregado secuencialmente sin exponer su representación subyacente.
- **Mediator:** Define un objeto que encapsula cómo interactúan un conjunto de objetos.
- **Memento:** Captura y externaliza el estado interno de un objeto sin violar su encapsulamiento.
- **Observer:** Define una dependencia uno a muchos entre objetos para que cuando un objeto cambie de estado, todos sus dependientes sean notificados y actualizados automáticamente.
- **State:** Permite que un objeto altere su comportamiento cuando su estado interno cambia.
- **Strategy:** Define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables.
- **Template Method:** Define el esqueleto de un algoritmo en una operación, diferiendo algunos pasos a subclases.
- **Visitor:** Representa una operación que se realizará en los elementos de una estructura de objeto.

Patrones GOF: Beneficios

- **Reutilización:** Proporcionan soluciones probadas y reutilizables.
- **Mantenibilidad:** Facilitan el mantenimiento y la evolución del código.
- **Comunicación:** Mejoran la comunicación entre desarrolladores al usar un lenguaje común para describir soluciones.

Patrones GOF: Beneficios

- **Reutilización:** Proporcionan soluciones probadas y reutilizables.
- **Mantenibilidad:** Facilitan el mantenimiento y la evolución del código.
- **Comunicación:** Mejoran la comunicación entre desarrolladores al usar un lenguaje común para describir soluciones.

Universidad Tecnológica del Perú

Patrones GOF: Cuando Usarlos

- Cuando se enfrentan a problemas recurrentes en el diseño de software.
- Cuando se busca una solución estandarizada y comprobada para un problema específico.

Universidad Tecnológica del Perú

Patrones GOF: Como Usarlos

- Identificar el problema específico en el diseño.
- Elegir el patrón adecuado que se ajuste a ese problema.
- Implementar el patrón según las directrices establecidas.

Clasificación del Patron según su proposito

1. **Patrones Creacionales:** Se ocupan de la creación de objetos y facilitan la instanciación de clases.

- **Ejemplos:** Singleton, Factory Method, Abstract Factory.
- **Descripción:** Manejan la complejidad de la creación de objetos al proporcionar mecanismos para crear instancias de manera controlada y flexible.

Clasificación del Patron según su proposito

2. **Patrones Estructurales:** Se centran en la composición de clases y objetos para formar estructuras más grandes.

- **Ejemplos:** Adapter, Composite, Decorator.
- **Descripción:** Facilitan la organización de las clases y objetos en estructuras que sean más fáciles de manejar y extender.

Clasificación del Patron según su proposito

3. **Patrones de Comportamiento:** Abordan la comunicación y la asignación de responsabilidades entre objetos.

- **Ejemplos:** Observer, Strategy, Command.
- **Descripción:** Ayudan a definir cómo los objetos interactúan y colaboran, y cómo se distribuyen las responsabilidades entre ellos.

Universidad Tecnológica del Perú

Especificación de Patrones de Diseño

1. Singleton: Asegura que una clase tenga solo una instancia y proporciona un punto global de acceso a ella.

```
public class Singleton {
    private static Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

Especificación de Patrones de Diseño

2. Factory Method: Define una interfaz para crear un objeto, pero deja que las subclases alteren el tipo de objeto que se creará.

```
interface Product {
    void use();
}

class ConcreteProductA implements Product {
    public void use() {
        System.out.println("Usando Producto A");
    }
}

class ConcreteProductB implements Product {
    public void use() {
        System.out.println("Usando Producto B");
    }
}
```

```
abstract class Creator {
    public abstract Product factoryMethod();
}

class ConcreteCreatorA extends Creator {
    public Product factoryMethod() {
        return new ConcreteProductA();
    }
}

class ConcreteCreatorB extends Creator {
    public Product factoryMethod() {
        return new ConcreteProductB();
    }
}
```

Especificación de Patrones de Diseño

3. Observer: Permite que un objeto notifique a otros objetos sobre cambios en su estado.

```
import java.util.ArrayList;
import java.util.List;

interface Observer {
    void update(String message);
}

class ConcreteObserver implements Observer {
    private String name;

    public ConcreteObserver(String name) {
        this.name = name;
    }

    public void update(String message) {
        System.out.println(name + " recibió el mensaje: " +
message);
    }
}
```

```
class Subject {
    private List<Observer> observers = new
ArrayList<>();

    public void addObserver(Observer observer) {
        observers.add(observer);
    }

    public void notifyObservers(String message) {
        for (Observer observer : observers) {
            observer.update(message);
        }
    }
}
```

Casos Practicos:

1. **Caso Práctico 1: Patrón Singleton - Configuración de Aplicación**

   Contexto: Imagina que estás desarrollando una aplicación que necesita una configuración global accesible en todo el sistema. Solo debe haber una instancia de la configuración para evitar inconsistencias y garantizar que todos los componentes usen los mismos valores.

   Problema: Sin un mecanismo adecuado, podrías terminar creando múltiples instancias de la clase de configuración, lo que podría llevar a errores y dificultades para mantener la configuración sincronizada

   Solución:
   Implementar……

Casos Practicos:

1. **Caso Práctico 2: Patrón Factory Method - Creación de Productos**

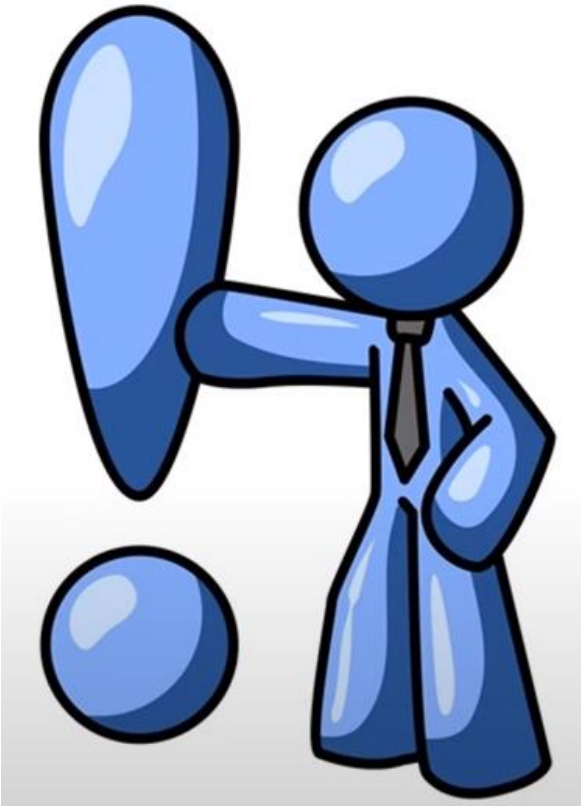
   Contexto: Estás desarrollando un sistema que necesita crear diferentes tipos de productos (por ejemplo, diferentes tipos de reportes: PDF y Word). El sistema debe decidir qué tipo de producto crear en tiempo de ejecución.

   Problema: Necesitas una manera flexible de crear productos sin tener que modificar el código cada vez que se añaden nuevos tipos de productos.

   Solución:
   Implementar……

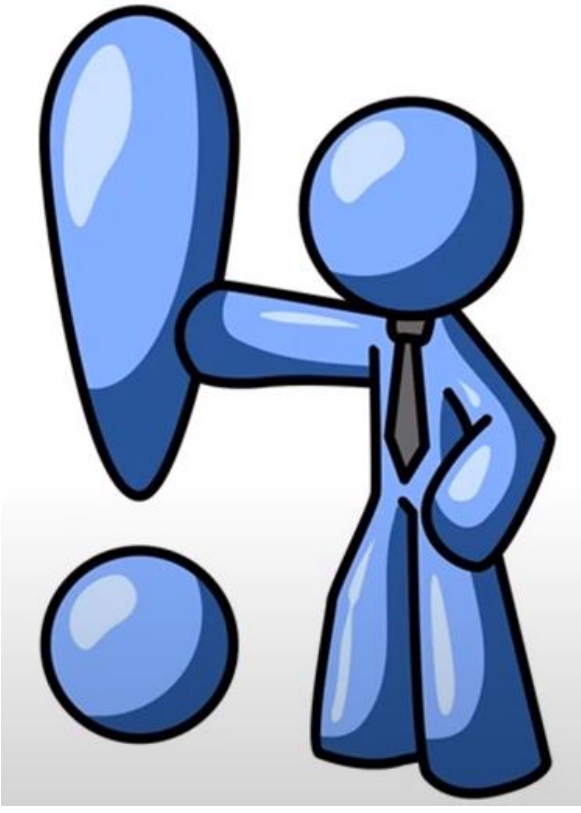
CONCLUSIONES

- **Introducción a los Patrones:** Los patrones de diseño son soluciones reutilizables a problemas comunes en el desarrollo de software, que mejoran la flexibilidad y mantenibilidad del código.
- **Patrones Arquitectónicos vs. Patrones de Diseño:**
  - **Arquitectónicos:** Definen la estructura general del sistema (ej. MVC).
  - **De Diseño:** Abordan problemas específicos dentro de un sistema (ej. Singleton).
- **Patrones GOF:** Los patrones del libro "Design Patterns" de los Gang of Four incluyen creacionales, estructurales y de comportamiento, ofreciendo soluciones estandarizadas a problemas recurrentes.



CONCLUSIONES

- **Clasificación según Propósito:**
  - **Creacionales:** Gestionan la creación de objetos (ej. Factory Method).
  - **Estructurales:** Organizan la composición de objetos (ej. Adapter).
  - **Comportamiento:** Definen la interacción entre objetos (ej. Observer).
- **Especificación de Patrones de Diseño:** Cada patrón se describe con su nombre, el problema que soluciona, la solución propuesta y ejemplos de implementación para facilitar su comprensión y uso.



Universidad Tecnológica del Perú

FACULTADAD DE INGENIERÍA

METACOGNICIÓN

1 ¿ QUÉ HE APRENDIDO ?

SER CONSCIENTE

2 ¿ CÓMO LO HE APRENDIDO ?

DESCRIBIR

3 ¿ QUÉ HA RESULTADO MÁS FÁCIL, MÁS DIFICIL O MÁS NOVEDOSO ?

EVALUAR

4 ¿ CÓMO LO PUEDO MEJORAR? ¿PARA QUÉ ME HA SERVIDO?

PLANIFICAR

Universidad Tecnológica del Perú

FACULTADAD DE INGENIERÍA

¡GRACIAS!

EN LA UTP SABEMOS QUE TODO ES POSIBLE